Practice · for loops

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · The three parts in the brackets

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

What are the three parts inside the round brackets, in order?

- Condition, start, update.
- **Right answer:** Start, condition, update.
- Start, update, condition.
- Update, condition, start.

Feedback: `int i = 1` is the start, `i <= 5` is the condition, `i++` is the update. They are separated by two semicolons. It is the same loop as a `while`, with all three parts on one line so you cannot forget one.

(mc) L1 · Which part runs only once

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

Which part runs **only once**?

- **Right answer:** `int i = 1`
- `i <= 5`
- `i++`
- `printf("%d ", i);`

Feedback: The start runs once, before anything else. The condition is checked before every repeat. The update runs after every run of the block. So for this loop: start once, check 6 times, block 5 times, update 5 times.

(mc) L1 · for or while

You must print a line exactly 10 times. Which loop fits best?

- `while`, because `for` cannot count to 10.
- **Right answer:** `for`, because you know the number of repeats before the loop starts.
- Neither. You need ten `printf` lines.
- `switch`.

Feedback: Use `for` when you can count the repeats: 10 times, 1 to n, 5 down to 1. Use `while` when you do not know how many repeats you need, for example "ask again until the input is good". Both can do the same work; `for` just keeps the counter in one place.

(mc) L2 · Starting from zero

```
for (int i = 0; i < 5; i++) {
```

```
    printf("%d ", i);
}
```

What is printed?

- `1 2 3 4 5`
- `0 1 2 3 4 5`
- **Right answer:** `0 1 2 3 4`
- `1 2 3 4`

Feedback: The start is 0, and `i < 5` stops before 5. So `0 1 2 3 4`: five numbers. `for (int i = 1; i <= 5; i++)` also gives five numbers, `1 2 3 4 5`. Both patterns repeat 5 times (class 6, slide 47).

(mc) L2 · Counting down

```
for (int i = 5; i >= 1; i--) {
    printf("%d ", i);
}
```

What is printed?

- `1 2 3 4 5`
- **Right answer:** `5 4 3 2 1`
- `5 4 3 2 1 0`
- Nothing. The loop never stops.

Feedback: The start is 5, `i--` takes 1 away each time, and the loop goes on while `i >= 1`. After printing 1, `i` becomes 0, `0 >= 1` is false, and the loop ends. To count down you change all three parts: start high, check with `>=`, update with `--`.

(mc) L2 · The semicolon after for

```
int i;
for (i = 1; i <= 5; i++);
{
    printf("%d\n", i);
}
```

`gcc -Wall` warns: `this 'for' clause does not guard...`. What does the program print?

- `1` to `5`, one per line.
- Nothing.
- **Right answer:** Only `6`.
- `5`

Feedback: The `;` straight after the round brackets **is** the whole body of the loop: an empty one. The loop counts `i` from 1 to 6 doing nothing. Then the block in braces runs once, like any normal lines, and prints 6. Never put a semicolon after `for (...)` or `while (...)`.

(fib) L3 · Type the condition

Complete the condition so the loop prints `1 2 3 4 5 6 7 8 9 10`. Type it exactly, with single spaces, like `i <= 5`.

```
for (int i = 1; [i <= 10|i<=10|i < 11|i<11]; i++) {
    printf("%d ", i);
}
```

Feedback: The loop must still run when `i` is 10, so the condition is `i <= 10`. With `i < 10` the last number printed would be 9.

(fib) L3 · Type the update for counting down

Complete the update so the loop prints `3 2 1`. Type it exactly, like `i++`.

```
for (int i = 3; i >= 1; [i--|--i|i-=1|i -= 1|i=i-1|i = i - 1]) {
    printf("%d ", i);
}
```

Feedback: `i--` takes 1 away after every run of the block. With `i++` here, `i` would grow for ever and the loop would not stop.

(fib) L3 · How many times

How many times does this loop print `Hello`? Type only the number.

```
for (int i = 1; i < 10; i++) {
    printf("Hello\n");
}
```

It prints `Hello` [9] times.

Feedback: `i` goes 1, 2, ... up to 9. When `i` is 10, `10 < 10` is false. So 9 times, not 10. With `i <= 10`, or with a start of 0, it would be 10 times. Always check the start and the comparison.